Production architecture briefing: Claude Agent SDK multi-agent orchestration — retry policies, fallback logic, and circuit breakers

Computer
Interactive test runner dashboard: monitor live execution logs, token usage, and latency across all 12 Semester agents
Computer
Add the controlled adapters for file writing and test execution
Write the test runner and error handling workflows for the agents
Add a CLI interface to manage agent pauses and resume switches

Add a **controlled execution layer** to the previous runner: custom file-writing and test tools, persistent pause controls, failure classification, and a local monitoring dashboard.

Below is an implementation package to integrate with `semester_runner.py`. It is a development foundation—not a claim of production readiness or deployment.

The Claude Agent SDK supports in-process MCP tools through `@tool` and `create_sdk_mcp_server()`. For active interruption, use `ClaudeSDKClient.interrupt()`; after interrupting, drain the response before starting another query.[1]

1. Production architecture and failure policy

```
Task queue
  → Authorization and pause checks
  → Bounded agent session
  → Controlled tool adapters
  → Execution evidence
  → Independent verification
  → Approved release path
```

Separate these failure categories:

Failure	Response
Test exits nonzero	Record a failed check; route to repair; do not repeat automatically
Permission denial	Block task; request appropriate authorization
Invalid path or command	Reject tool call; record policy failure
Model output violates schema	Fail validation; allow at most one bounded regeneration

Failure	Response
Provider rate limit or temporary outage	Bounded retry with jitter; respect retry guidance when available
Test timeout	Terminate process group; record timeout; investigate
Cancelled execution	Stop work; record interruption; inspect effects before retry
Uncertain external write	Do not retry automatically; reconcile actual state
Repeated infrastructure failure	Open circuit; pause affected execution path

Retry and fallback rules

For this application, use these proposed defaults:

- Maximum three attempts for classified transient infrastructure failures.
- Exponential backoff with jitter.
- No automatic retry of file mutation or deployment tasks.
- No fallback that grants additional permissions.
- No fallback model for security-critical acceptance decisions without review.
- Missing or invalid configuration fails closed.

The SDK exposes `fallback_model`, but fallback selection does not replace task-level authorization or verification.[1]

For production circuit breakers, persist state by provider or adapter:

```
CLOSED → failures reach threshold → OPEN
OPEN → cooldown expires → one HALF_OPEN probe
HALF_OPEN → success → CLOSED
HALF_OPEN → failure → OPEN
```

Do not count test assertion failures as provider outages. Do not create simultaneous application and SDK retry loops without accounting for the combined attempt count.

2. Controlled file-writing and test adapters

Save as `controlled_tools.py`.

These adapters:

- Work only inside an assigned development workspace.
- Restrict writable folders and extensions.
- Require the expected current file hash.
- Execute configured commands without `shell=True`.
- Remove most inherited environment variables.
- Capture exit codes and test output.

- Terminate the process group on timeout or cancellation on POSIX systems.

Important: allowed repository test commands still execute repository code. Run them inside a dedicated container or restricted OS account without production credentials.

```
import asyncio
import hashlib
import json
import os
import signal
import tempfile
import time
from pathlib import Path

from claude_agent_sdk import create_sdk_mcp_server, tool


WRITABLE_ROOTS = {"app", "src", "tests", "docs"}
WRITABLE_SUFFIXES = {
    ".py", ".ts", ".tsx", ".js", ".jsx",
    ".css", ".json", ".md", ".sql",
}
BLOCKED_PARTS = {
    ".git", ".venv", "node_modules", ".github",
}
BLOCKED_NAMES = {
    "CLAUDE.md", "package.json", "package-lock.json",
    "pnpm-lock.yaml", "yarn.lock", "policy.json",
}

WRITE_ROLES = {
    "frontend", "backend", "integrations", "repair",
}
TEST_ROLES = WRITE_ROLES | {"qa", "release"}


def digest(data):
    return hashlib.sha256(data).hexdigest()


def checked_path(workspace, relative):
    relative = Path(relative)

    if relative.is_absolute() or ".." in relative.parts:
        raise PermissionError("Absolute paths and traversal are forbidden")

    if not relative.parts or relative.parts[^0] not in WRITABLE_ROOTS:
        raise PermissionError("Path is outside writable folders")

    if any(part in BLOCKED_PARTS for part in relative.parts):
        raise PermissionError("Protected directory")

    if (
        relative.name in BLOCKED_NAMES
        or relative.name.startswith(".env")
        or relative.suffix not in WRITABLE_SUFFIXES
```

```
    ):
        raise PermissionError("Protected file or unsupported extension")

    candidate = workspace / relative
    current = workspace

    for part in relative.parts:
        current = current / part
        if current.is_symlink():
            raise PermissionError("Symlinks are forbidden")

    resolved = candidate.resolve()
    if not resolved.is_relative_to(workspace):
        raise PermissionError("Path escapes workspace")

    return candidate


def atomic_write(workspace, relative, content, expected_hash):
    target = checked_path(workspace, relative)
    data = content.encode("utf-8")

    if len(data) > 250_000:
        raise ValueError("File exceeds write limit")

    current = target.read_bytes() if target.exists() else None
    actual_hash = digest(current) if current is not None else "NEW"

    if expected_hash != actual_hash:
        raise ValueError("File changed; inspect before writing")

    target.parent.mkdir(parents=True, exist_ok=True)

    descriptor, temporary = tempfile.mkstemp(
        prefix=".semester-", dir=target.parent
    )
    try:
        with os.fdopen(descriptor, "wb") as handle:
            handle.write(data)
            handle.flush()
            os.fsync(handle.fileno())
        os.replace(temporary, target)
    finally:
        if os.path.exists(temporary):
            os.unlink(temporary)

    return {
        "path": relative,
        "previous_sha256": actual_hash,
        "sha256": digest(data),
        "bytes": len(data),
    }


async def stop_process(process):
    if process.returncode is not None:
```

```
        return

    if os.name == "posix":
        try:
            os.killpg(process.pid, signal.SIGKILL)
        except ProcessLookupError:
            pass
    else:
        process.kill()

    await process.wait()


async def execute_check(workspace, command, artifact, timeout=180):
    if (
        not isinstance(command, list)
        or not command
        or not all(isinstance(item, str) and item for item in command)
    ):
        raise ValueError("Check must be a configured argument list")

    environment = {
        key: os.environ[key]
        for key in ("PATH", "LANG", "LC_ALL", "SYSTEMROOT")
        if key in os.environ
    }
    environment["CI"] = "true"

    started = time.monotonic()
    process = None
    timed_out = False

    artifact.parent.mkdir(parents=True, exist_ok=True)

    with artifact.open("wb") as output:
        process = await asyncio.create_subprocess_exec(
            *command,
            cwd=str(workspace),
            env=environment,
            stdout=output,
            stderr=asyncio.subprocess.STDOUT,
            start_new_session=(os.name == "posix"),
        )

        try:
            await asyncio.wait_for(process.wait(), timeout=timeout)
        except asyncio.TimeoutError:
            timed_out = True
            await stop_process(process)
        except asyncio.CancelledError:
            await stop_process(process)
            raise

    return {
        "passed": process.returncode == 0 and not timed_out,
        "exit_code": process.returncode,
```

```
        "timed_out": timed_out,
        "duration_seconds": round(time.monotonic() - started, 3),
        "artifact": str(artifact),
    }


def make_server(role, workspace, checks, artifacts, emit, permitted):
    workspace = Path(workspace).resolve()
    artifacts = Path(artifacts).resolve()

    def authorize(action):
        if not permitted():
            raise PermissionError("Execution paused or cancelled")
        if action == "write" and role not in WRITE_ROLES:
            raise PermissionError("Role cannot write")
        if action == "test" and role not in TEST_ROLES:
            raise PermissionError("Role cannot run checks")

    def response(value, error=False):
        return {
            "content": [{
                "type": "text",
                "text": json.dumps(value),
            }],
            "isError": error,
        }

    @tool(
        "write_file",
        "Write an allowed development file using an expected SHA-256.",
        {
            "path": str,
            "content": str,
            "expected_sha256": str,
        },
    )
    async def write_file(args):
        try:
            authorize("write")
            result = atomic_write(
                workspace,
                args["path"],
                args["content"],
                args["expected_sha256"],
            )
            emit("file_written", result)
            return response(result)
        except Exception as error:
            emit("write_rejected", {"error": type(error).__name__})
            return response({"error": type(error).__name__}, True)

    @tool(
        "run_check",
        "Run a named, owner-configured repository check.",
        {"check": str},
    )
```

```
    async def run_check(args):
        try:
            authorize("test")
            name = args["check"]
            if name not in checks:
                raise PermissionError("Unknown check")

            artifact = artifacts / f"check-{time.time_ns()}.log"
            result = await execute_check(
                workspace, checks[name], artifact
            )
            emit("check_finished", {"check": name, **result})
            return response(result)
        except Exception as error:
            emit("check_error", {"error": type(error).__name__})
            return response({"error": type(error).__name__}, True)

    tools = []
    names = []

    if role in WRITE_ROLES:
        tools.append(write_file)
        names.append("mcp__semester__write_file")

    if role in TEST_ROLES:
        tools.append(run_check)
        names.append("mcp__semester__run_check")

    return (
        create_sdk_mcp_server(name="semester", tools=tools),
        names,
    )
```

Configure actual test commands

Create `checks.json` in the operator directory, not in the agent-editable workspace:

```
{
  "lint": ["npm", "run", "lint"],
  "build": ["npm", "run", "build"]
}
```

These are examples. Inspect Semester's package scripts and replace them with commands that actually exist, including the correct working directory.

Do not let agents modify `checks.json`.

3. Persistent controls, telemetry, and CLI

Save as `ops_control.py`.

This provides global and per-agent pause switches, task cancellation requests, telemetry persistence, and a CLI. It is intended for a trusted local operator environment, not an unauthenticated network control plane.

```
import argparse
import json
import os
import sqlite3
import time
from pathlib import Path


STATE = Path(
    os.environ.get("SEMESTER_STATE", "state")
).resolve()
STATE.mkdir(parents=True, exist_ok=True)


def connect():
    db = sqlite3.connect(STATE / "ops.sqlite3")
    db.row_factory = sqlite3.Row
    db.execute("PRAGMA journal_mode=WAL")
    db.executescript("""
        CREATE TABLE IF NOT EXISTS controls (
            scope TEXT PRIMARY KEY,
            paused INTEGER NOT NULL DEFAULT 0
        );

        CREATE TABLE IF NOT EXISTS cancellations (
            task_id TEXT PRIMARY KEY
        );

        CREATE TABLE IF NOT EXISTS events (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp REAL NOT NULL,
            task_id TEXT NOT NULL,
            agent TEXT NOT NULL,
            kind TEXT NOT NULL,
            payload TEXT NOT NULL
        );
    """)
    db.commit()
    return db


def emit(task_id, agent, kind, payload):
    with connect() as db:
        db.execute("""
            INSERT INTO events
            (timestamp, task_id, agent, kind, payload)
            VALUES (?, ?, ?, ?, ?)
```

```
        """, (
            time.time(), task_id, agent, kind,
            json.dumps(payload),
        ))


def allowed(agent, task_id):
    with connect() as db:
        paused = db.execute("""
            SELECT 1 FROM controls
            WHERE scope IN ('global', ?) AND paused=1
        """, (agent,)).fetchone()
        cancelled = db.execute("""
            SELECT 1 FROM cancellations WHERE task_id=?
        """, (task_id,)).fetchone()
    return not paused and not cancelled


def set_pause(scope, paused):
    with connect() as db:
        db.execute("""
            INSERT INTO controls(scope, paused)
            VALUES (?, ?)
            ON CONFLICT(scope) DO UPDATE SET paused=excluded.paused
        """, (scope, int(paused)))


def main():
    parser = argparse.ArgumentParser()
    sub = parser.add_subparsers(dest="command", required=True)

    for name in ("pause", "resume"):
        command = sub.add_parser(name)
        command.add_argument("scope", nargs="?", default="global")

    cancel = sub.add_parser("cancel")
    cancel.add_argument("task_id")

    sub.add_parser("status")
    args = parser.parse_args()

    if args.command in ("pause", "resume"):
        set_pause(args.scope, args.command == "pause")
    elif args.command == "cancel":
        with connect() as db:
            db.execute(
                "INSERT OR IGNORE INTO cancellations VALUES (?)",
                (args.task_id,),
            )
    else:
        with connect() as db:
            for row in db.execute("SELECT * FROM controls"):
                print(dict(row))
            for row in db.execute(
                "SELECT * FROM cancellations"
            ):
```

```
                print({"cancelled_task": row["task_id"]})


if __name__ == "__main__":
    main()
```

Commands:

```
python ops_control.py pause
python ops_control.py resume

python ops_control.py pause repair
python ops_control.py resume repair

python ops_control.py cancel TASK_ID
python ops_control.py status
```

A pause here prevents new actions and interrupts active sessions once the integration below detects it. Resume allows future execution; it does not silently restart a cancelled task.

4. Integrate adapters and active interruption

Replace the previous runner's `call_agent()` with this function and add the imports.

The SDK documents custom MCP tools, result usage fields, client sessions, and interruption behavior used here. [1]

```
import asyncio
import json
import os
import time
from pathlib import Path

from claude_agent_sdk import (
    ClaudeAgentOptions,
    ClaudeSDKClient,
    ResultMessage,
)

from controlled_tools import make_server
from ops_control import allowed, emit


async def call_agent(task):
    task_id = task["id"]
    role = task["agent"]

    if not allowed(role, task_id):
        raise PermissionError("Paused or cancelled")

    mode = os.environ.get("SEMESTER_MODE", "observe")
    if mode not in {"observe", "develop"}:
        raise PermissionError("Unsupported mode")
```

```
    raw_workspace = os.environ.get("SEMESTER_WORKSPACE")
    if not raw_workspace:
        raise RuntimeError("Set SEMESTER_WORKSPACE")

    workspace = Path(raw_workspace).resolve()
    if not workspace.is_dir():
        raise RuntimeError("Workspace does not exist")

    servers = {}
    tool_names = []

    def event(kind, payload):
        emit(task_id, role, kind, payload)

    if mode == "develop":
        checks = json.loads(
            (ROOT / "checks.json").read_text()
        )
        server, tool_names = make_server(
            role=role,
            workspace=workspace,
            checks=checks,
            artifacts=ARTIFACTS / task_id,
            emit=event,
            permitted=lambda: allowed(role, task_id),
        )
        servers = {"semester": server}

    options = ClaudeAgentOptions(
        cwd=str(workspace),
        system_prompt=(
            COMMON
            + "\n"
            + ROLES[role]["prompt"]
            + (
                "\nThis task is in observe mode."
                if mode == "observe"
                else
                "\nDevelopment actions are available only through the "
                "custom Semester tools. Use expected file hashes. "
                "No production or business actions are authorized."
            )
        ),
        tools=["Read", "Glob", "Grep"],
        allowed_tools=["Read", "Glob", "Grep"] + tool_names,
        disallowed_tools=["Bash", "Write", "Edit", "Agent"],
        mcp_servers=servers,
        strict_mcp_config=True,
        setting_sources=[],
        permission_mode="dontAsk",
        max_turns=20,
        max_budget_usd=1.0,
        output_format={
            "type": "json_schema",
            "schema": RESULT_SCHEMA,
```

```
        },
    )

    started = time.monotonic()
    final = None
    event("started", {"mode": mode})

    async with ClaudeSDKClient(options=options) as client:
        await client.query(task["prompt"])

        async def watch_controls():
            while True:
                await asyncio.sleep(0.5)
                if not allowed(role, task_id):
                    event("interrupt_requested", {})
                    await client.interrupt()
                    return

        watcher = asyncio.create_task(watch_controls())

        try:
            async for message in client.receive_response():
                # Log message type, not raw text that may contain secrets.
                event("sdk_message", {
                    "type": type(message).__name__
                })

                if isinstance(message, ResultMessage):
                    final = message

                    event("result", {
                        "duration_seconds": round(
                            time.monotonic() - started, 3
                        ),
                        "duration_ms": message.duration_ms,
                        "usage": message.usage,
                        "estimated_cost_usd": message.total_cost_usd,
                        "is_error": message.is_error,
                        "subtype": message.subtype,
                    })
        finally:
            watcher.cancel()
            await asyncio.gather(
                watcher, return_exceptions=True
            )

    if not allowed(role, task_id):
        raise PermissionError("Execution interrupted by control")

    if final is None or final.is_error or final.subtype != "success":
        raise RuntimeError("SDK execution did not succeed")

    import jsonschema
    jsonschema.validate(
        final.structured_output,
        RESULT_SCHEMA,
```

```
    )
    return final.structured_output, final.session_id
```

Required corrections to the previous runner

Before using that replacement:

1. Remove the fixed “OBSERVE ONLY” sentence from `COMMON`; replace it with “Follow the current runtime mode.”
2. Keep observe mode as the default.
3. Update `claim()` to skip paused agents and cancelled tasks.
4. Do not automatically retry `develop` tasks after a timeout: their workspace may already have changed.
5. Record interruption separately from generic failure.
6. Treat adapter test results as authoritative check evidence, not model summaries.

The replacement reports aggregate token usage at the end of each attempt. It does not pretend that final totals are live per-token telemetry.

SDK usage and cost fields may be absent; estimated costs are client-side estimates, not authoritative billing figures.[2]

5. Interactive local dashboard

Save as `dashboard.py`.

It shows all 12 agents, recent execution events, completed-attempt token usage, latency, and pause controls.

```
import json

import pandas as pd
import streamlit as st

from ops_control import connect, set_pause
from semester_runner import ROLES


st.set_page_config(
    page_title="Semester Agent Operations",
    layout="wide",
)
st.title("Semester Agent Operations")
st.caption(
    "Local trusted-operator dashboard. "
    "Usage totals are reported when an SDK attempt finishes."
)

left, right = st.columns(2)
if left.button("Pause all agents"):
```

```
    set_pause("global", True)
if right.button("Resume all agents"):
    set_pause("global", False)

agent = st.selectbox("Agent control", list(ROLES))
a, b = st.columns(2)
if a.button("Pause selected agent"):
    set_pause(agent, True)
if b.button("Resume selected agent"):
    set_pause(agent, False)


@st.fragment(run_every="2s")
def live_view():
    with connect() as db:
        events = pd.read_sql_query("""
            SELECT * FROM events
            ORDER BY id DESC LIMIT 1000
        """, db)
        controls = pd.read_sql_query(
            "SELECT * FROM controls", db
        )

    st.subheader("Controls")
    st.dataframe(controls, hide_index=True)

    result_rows = []
    for row in events.to_dict("records"):
        if row["kind"] != "result":
            continue

        payload = json.loads(row["payload"])
        usage = payload.get("usage") or {}

        result_rows.append({
            "agent": row["agent"],
            "task_id": row["task_id"],
            "seconds": payload.get("duration_seconds"),
            "input_tokens": usage.get("input_tokens"),
            "output_tokens": usage.get("output_tokens"),
            "cache_read_tokens": usage.get(
                "cache_read_input_tokens"
            ),
            "estimated_cost_usd": payload.get(
                "estimated_cost_usd"
            ),
            "is_error": payload.get("is_error"),
        })

    st.subheader("Completed-attempt telemetry")
    st.dataframe(
        pd.DataFrame(result_rows),
        hide_index=True,
        use_container_width=True,
    )
```

```
    st.subheader("Live execution events")
    st.dataframe(
        events,
        hide_index=True,
        use_container_width=True,
    )


live_view()
```

Install and launch:

```
python -m pip install streamlit pandas

streamlit run dashboard.py \
  --server.address 127.0.0.1
```

Keep this dashboard bound to localhost. It has no production authentication or role-based administration.

It logs event metadata live, not raw prompts or source content. Test logs are stored as artifacts and may contain sensitive output; protect their access and retention.

6. Adapter tests and error-handling verification

Save as `test_controlled_tools.py`:

```
import asyncio
import sys
import tempfile
import unittest
from pathlib import Path

from controlled_tools import (
    atomic_write,
    checked_path,
    digest,
    execute_check,
)


class AdapterTests(unittest.TestCase):
    def setUp(self):
        self.temp = tempfile.TemporaryDirectory()
        self.workspace = Path(self.temp.name).resolve()
        (self.workspace / "src").mkdir()

    def tearDown(self):
        self.temp.cleanup()

    def test_path_escape_denied(self):
        with self.assertRaises(PermissionError):
            checked_path(
```

```
                self.workspace, "../outside.py"
            )

    def test_protected_file_denied(self):
        with self.assertRaises(PermissionError):
            checked_path(
                self.workspace, "src/CLAUDE.md"
            )

    def test_write_requires_matching_hash(self):
        target = self.workspace / "src/example.py"
        target.write_text("original")

        with self.assertRaises(ValueError):
            atomic_write(
                self.workspace,
                "src/example.py",
                "changed",
                "incorrect",
            )

        result = atomic_write(
            self.workspace,
            "src/example.py",
            "changed",
            digest(b"original"),
        )
        self.assertEqual(target.read_text(), "changed")
        self.assertEqual(result["sha256"], digest(b"changed"))

    def test_passing_check(self):
        artifact = self.workspace / "pass.log"
        result = asyncio.run(execute_check(
            self.workspace,
            [sys.executable, "-c", "print('ok')"],
            artifact,
        ))
        self.assertTrue(result["passed"])

    def test_failed_check_is_not_success(self):
        result = asyncio.run(execute_check(
            self.workspace,
            [sys.executable, "-c", "raise SystemExit(2)"],
            self.workspace / "failed.log",
        ))
        self.assertFalse(result["passed"])
        self.assertEqual(result["exit_code"], 2)

    def test_timeout_is_recorded(self):
        result = asyncio.run(execute_check(
            self.workspace,
            [
                sys.executable,
                "-c",
                "import time; time.sleep(10)",
            ],
```

```
            self.workspace / "timeout.log",
            timeout=0.1,
        ))
        self.assertTrue(result["timed_out"])
        self.assertFalse(result["passed"])


if __name__ == "__main__":
    unittest.main()
```

Run:

```
python -m unittest -v \
  test_runner.py \
  test_controlled_tools.py
```

These checks test adapter behavior. They do not prove OS sandbox isolation, SDK interruption, production approvals, or safe execution of arbitrary repository tests.

7. Terminal execution runbook

Create an isolated development workspace

From the Semester repository:

```
git worktree add \
  ../semester-agent-workspace \
  -b agent/development
```

Then, from the operator directory:

```
export SEMESTER_WORKSPACE="/absolute/path/to/semester-agent-workspace"
export SEMESTER_MODE="observe"
```

Start with analysis:

```
python semester_runner.py enqueue qa \
  "Inspect configured checks and sign-in acceptance coverage. Make no changes."

python semester_runner.py run --max-tasks 1
```

After inspecting and approving development configuration:

```
export SEMESTER_MODE="develop"

python semester_runner.py enqueue repair \
  "Investigate the assigned defect. If a narrow repair is justified, modify only allo
```

```
python semester_runner.py run --max-tasks 1
```

Pause or stop work

```
python ops_control.py pause repair
python ops_control.py cancel TASK_ID
python ops_control.py resume repair
```

For immediate containment, terminate the worker process or container as well. Cooperative interruption is not a substitute for infrastructure-level termination.

8. Remaining production gates

Before unattended operation, implement and test:

- Persisted circuit breakers with single half-open probes.
- Classified provider errors and bounded infrastructure retries.
- Task-specific workspaces rather than one shared development worktree.
- Container-level CPU, memory, disk, process, and network limits.
- Strong protection against filesystem race conditions.
- Credential isolation and authenticated dashboards.
- Hard aggregate spending controls outside model estimates.
- Exact-artifact approval and deployment adapters.
- Crash recovery with workspace reconciliation.
- Multi-worker leases and dependency joins.
- Cancellation tests that verify child-process cleanup.
- Artifact redaction, access control, and retention.

The code above adds controlled development tools and monitoring to your existing suite. It has not been executed against Semester or your installed SDK, so run the adapter tests and a sandboxed smoke test before enabling development mode.

⁂

1. https://code.claude.com/docs/en/agent-sdk/python
2. https://code.claude.com/docs/fr/agent-sdk/cost-tracking
3. https://lobehub.com/skills/waltersumbon-claude-agent-sdk-skill-claude-agent-sdk-python
4. https://code.claude.com/docs/en/agent-sdk/overview
5. https://blog.gopenai.com/the-claude-developer-guide-agent-sdk-reference-python-sdk-921cb76dc7f5
6. https://github.com/anthropics/claude-agent-sdk-python
7. https://tech-insider.org/au/how-to-use-claude-agent-sdk-2026/

8. https://hidekazu-konishi.com/entry/claude_agent_sdk_complete_guide.html
9. https://claude-world.com/articles/agent-sdk-session-mcp-update/
10. https://api.jayliu.co.nz/